

Prueba Técnica: Científico de Datos en NEQUI

Solución para Identificar Mala Práctica Transaccional



Alejandro Moncayo Riascos

Enero de 2025

Medellín

Introducción

El propósito de este proyecto es desarrollar un producto de datos para identificar transacciones que evidencian comportamientos de Mala Práctica Transaccional, específicamente casos de Fraccionamiento Transaccional. Este fenómeno se caracteriza por dividir una transacción en múltiples operaciones de menor monto realizadas dentro de una ventana de 24 horas, asociadas a un mismo cliente o cuenta. Este análisis permite detectar patrones sospechosos que pueden implicar un uso indebido de los canales financieros.

El enfoque combina técnicas analíticas, estadísticas y heurísticas para ofrecer una solución robusta y operativa, basada en datos reales y sintéticos con distribuciones representativas.

Paso 2: Explorar y evaluar los datos, el EDA.

Metodología

La solución se estructuró en varias etapas, desde la exploración de datos hasta la identificación de patrones sospechosos, detalladas a continuación. En esta sección se describe cómo se procesaron y analizaron los datos para garantizar que se cumplieran las condiciones necesarias para identificar prácticas de fraccionamiento transaccional. Este enfoque combina técnicas de manipulación y análisis de datos utilizando herramientas distribuidas, asegurando precisión y escalabilidad.

A. Preparación de Datos

Se cargaron dos archivos en formato Parquet (sample_data_0006_part_00.parquet y sample_data_0007_part_00.parquet) utilizando Dask. Este formato optimiza el almacenamiento y permite procesar datos de forma distribuida:

- **_id**: Identificador único de la transacción.
- **merchant_id**: Código único del comercio.
- **subsidiary**: Código único de la sucursal.
- **transaction_date**: Fecha de la transacción.
- **account_number**: Número de cuenta involucrada.
- **user_id**: Identificador único del usuario dueño de la cuenta.
- **transaction_amount**: Monto de la transacción.
- **transaction_type**: Naturaleza de la transacción (CRÉDITO o DÉBITO).

Integración y limpieza: Los datos de múltiples fuentes en formato Parquet fueron consolidados usando Dask, asegurando la eliminación de duplicados por el identificador único _id.

Conversión de tipos: Se transformaron las columnas transaction_date y transaction_amount a formatos adecuados (datetime64 y float, respectivamente). También se creó una nueva columna day_bucket para agrupar transacciones por día.

B. Generación de Agregados por Usuario y Cuenta

Se diseñó un pipeline para agrupar las transacciones por user_id, account_number, y day_bucket, calculando:

- `rolling_count`: Número total de transacciones realizadas en el día.
- `rolling_sum`: Suma acumulada de los montos de las transacciones.
- `rolling_std`: Desviación estándar de los montos de las transacciones

C. Exportación de Resultados Parciales y Consolidación

Los archivos intermedios generados durante el proceso, como los denominados `grouped_results_part_X.csv`, contienen particiones de datos procesados y organizados por usuario (`user_id`), cuenta (`account_number`), y ventanas temporales de 24 horas (`day_bucket`). En estos archivos se calculan métricas esenciales como `rolling_count`, que indica el número de transacciones realizadas por un usuario o cuenta en ese período; `rolling_sum`, que refleja el monto total acumulado en la ventana de 24 horas; y `rolling_std`, que mide la desviación estándar de los montos transaccionales, ayudando a identificar posibles anomalías.

Posteriormente, estos archivos parciales se consolidan en un único archivo denominado `grouped_results_combined.csv`, que reúne la totalidad de los datos procesados de manera estructurada y lista para realizar análisis más detallados. Este archivo consolidado se utiliza como base para calcular métricas adicionales y aplicar las reglas necesarias para detectar posibles casos de fraccionamiento transaccional.

La identificación de transacciones sospechosas se realiza aplicando reglas predefinidas. Una de estas reglas establece que el `rolling_count` debe ser mayor a 1, lo que implica más de una transacción en la ventana de 24 horas por usuario o cuenta. Adicionalmente, se considera un umbral para el `rolling_sum`, definido estadísticamente (como el percentil 99%), para identificar montos acumulados que sean inusualmente altos y potencialmente sospechosos. Como resultado de este análisis, se genera el archivo `fraccionamiento_detectado.csv`, que detalla las transacciones que cumplen con estas condiciones. Este archivo incluye información clave como los identificadores de usuario (`user_id`), cuenta (`account_number`), fechas (`day_bucket`), y las métricas `rolling_count` y `rolling_sum`.

D. Identificación de Transacciones Sospechosas

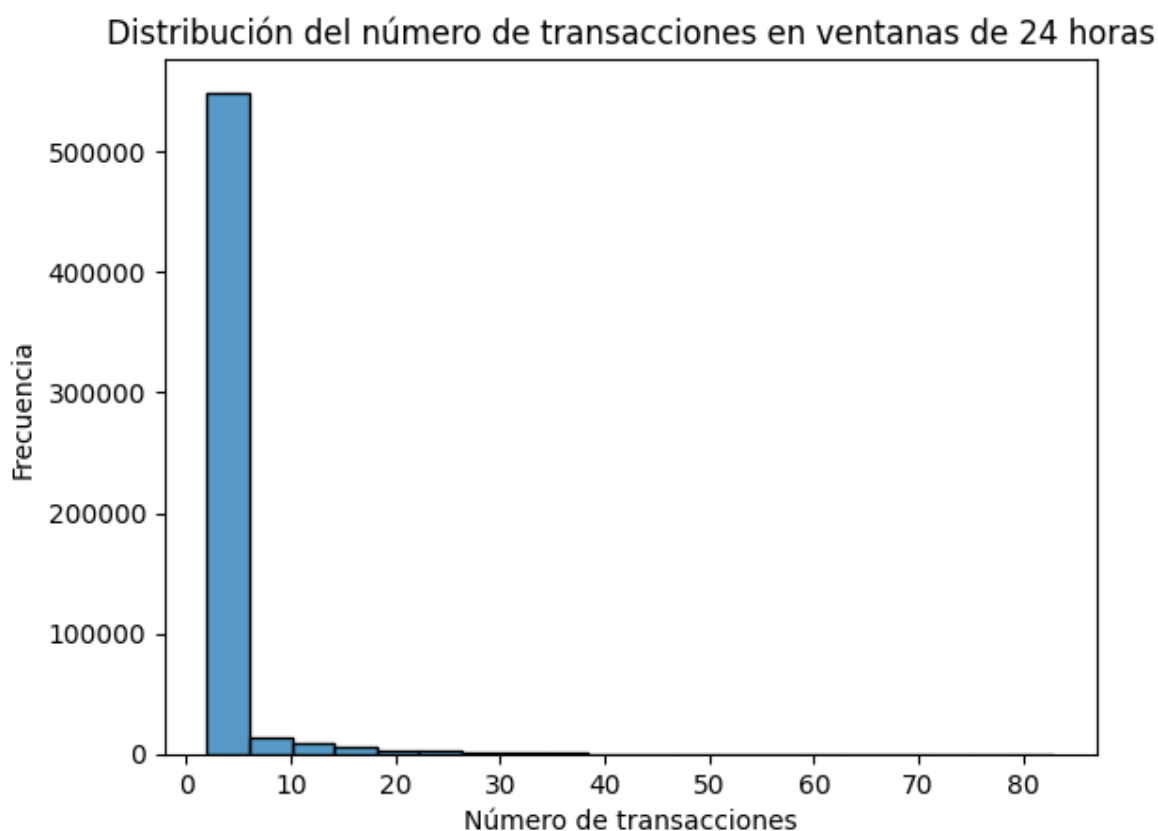
Se definieron criterios específicos para detectar prácticas de fraccionamiento transaccional:

- **Condición 1:** Más de una transacción (`rolling_count > 1`).
- **Condición 2:** Agrupadas en una misma ventana de 24 horas (`day_bucket`).

RESULTADOS

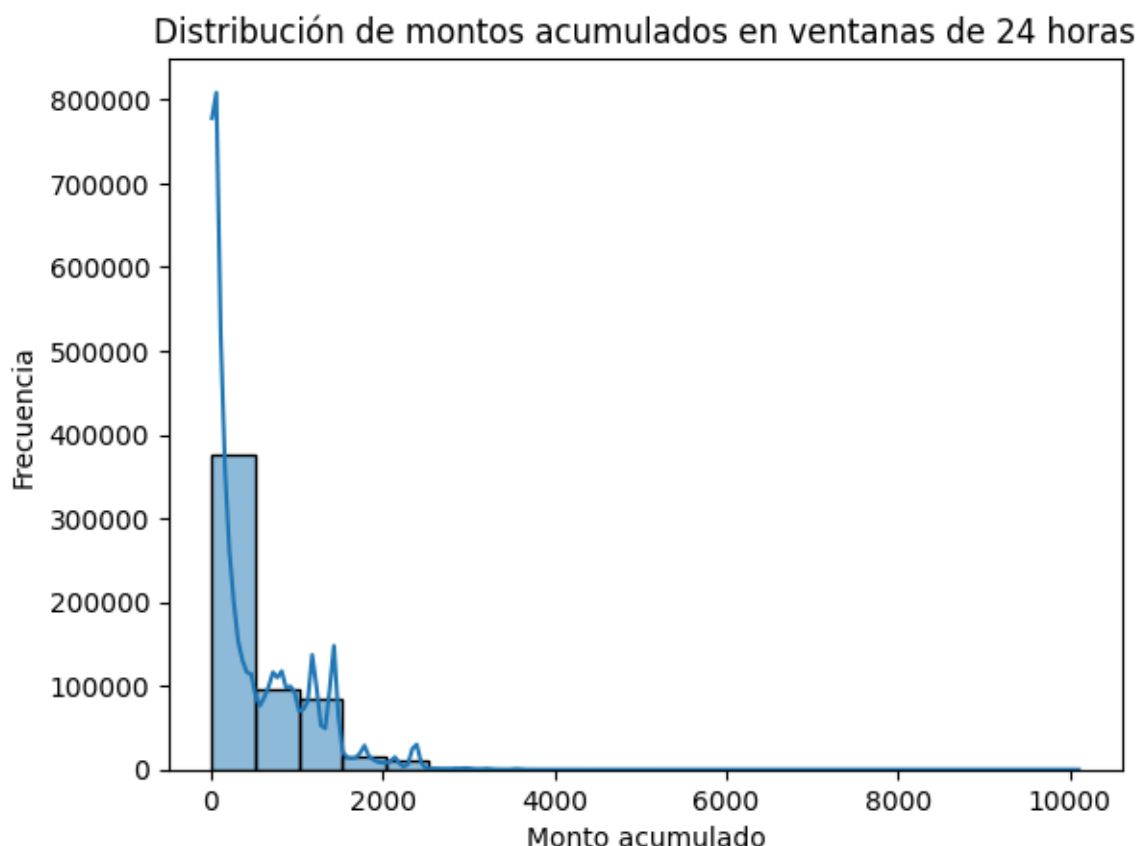
El análisis de las gráficas y resultados obtenidos en el proceso permite inferir patrones clave sobre el comportamiento transaccional en ventanas de 24 horas, así como identificar transacciones sospechosas de fraccionamiento. A continuación, se presentan las interpretaciones y conclusiones principales:

La distribución del número de transacciones en ventanas de 24 horas, mostrada en la primera gráfica, revela que la mayoría de los usuarios realizan menos de 10 transacciones diarias, lo cual está alineado con el comportamiento típico esperado en sistemas financieros. Sin embargo, hay casos aislados donde el número de transacciones supera significativamente este umbral, alcanzando hasta 80 transacciones en un solo día. Este comportamiento anómalo podría ser indicativo de una estrategia de fraccionamiento deliberado.



La segunda gráfica, que ilustra la **distribución de los montos acumulados** en estas mismas ventanas, muestra una concentración predominante de montos bajos, con una clara disminución a medida que los valores acumulados aumentan. Sin embargo, se observan picos intermitentes en montos superiores, lo cual sugiere que ciertos usuarios acumulan

transacciones significativas en un corto período de tiempo. Esto respalda la hipótesis de fraccionamiento, ya que estos montos podrían corresponder a valores de transacciones originales divididas en partes más pequeñas.



Del conjunto de transacciones analizadas, se detectaron **586,914 transacciones sospechosas** de fraccionamiento, asociadas a **293,138 usuarios únicos**. Estos resultados se basan en la condición de que los usuarios hayan realizado más de una transacción dentro de una ventana de 24 horas, acumulando montos significativos que podrían ser producto de prácticas indebidas. Las métricas clave, como `rolling_count` y `rolling_sum`, fueron fundamentales para identificar estos casos. Por ejemplo, un usuario específico presentó 2 transacciones en un día con un monto acumulado de 1,070.00, claramente fuera del comportamiento promedio.

La capacidad de identificar usuarios sospechosos y detallar sus patrones de transacciones proporciona un insumo crucial para sistemas de monitoreo financiero. Estas anomalías pueden ser priorizadas para revisión manual o análisis automatizado adicional utilizando modelos de clasificación.

Paso 3 Definir el modelo analítico

1. Modelado Analítico

El flujo de datos diseñado para identificar transacciones sospechosas de fraccionamiento transaccional está compuesto por varias etapas bien definidas, asegurando tanto la eficiencia operativa como la precisión analítica.

En la primera etapa, la ingesta de datos se parte de fuentes como sistemas de Core Bancario o plataformas de pago, donde se recolectan transacciones con atributos relevantes como el identificador único del usuario (`user_id`), número de cuenta (`account_number`), fecha y hora de la transacción (`transaction_date`), y el monto transaccionado (`transaction_amount`). Este paso incluye la validación inicial de los datos, eliminando registros duplicados, manejando formatos inconsistentes y garantizando la calidad de la información antes de procesarla. Los datos se pueden ingerir en tiempo real, para detectar posibles fraccionamientos de manera inmediata, o mediante procesamiento por lotes, para análisis más detallados y consolidados. Herramientas como Apache Kafka y Apache Nifi son fundamentales en esta etapa, ya que permiten manejar la transmisión y transformación de los datos con gran eficacia.

El procesamiento es la siguiente etapa crítica del flujo. Aquí, las transacciones se agrupan utilizando una ventana temporal de 24 horas, lo que permite analizar todas las operaciones realizadas por un cliente específico dentro de ese periodo. Este análisis se realiza considerando no solo el cliente (`user_id`) sino también las cuentas asociadas (`account_number`), lo que garantiza que se incluyan todos los movimientos relacionados con la misma cuenta. En este paso, se calculan métricas clave como el número de transacciones en la ventana (`rolling_count`), la suma acumulada de los montos transaccionados (`rolling_sum`), y la desviación estándar de los montos (`rolling_std`). Estas métricas son calculadas usando herramientas distribuidas como Dask o Apache Spark, que permiten procesar grandes volúmenes de datos con rapidez y escalabilidad.

Una vez procesados los datos, se procede a la validación y filtrado. En esta etapa, se identifican posibles casos de fraccionamiento transaccional. Esto incluye transacciones que cumplen con la condición de haber ocurrido más de una vez dentro de la misma ventana de 24 horas. Adicionalmente, se pueden aplicar umbrales sobre el monto acumulado (`rolling_sum`) para priorizar las transacciones de mayor relevancia, con base en análisis estadísticos como el percentil 99 de los montos transaccionados. Para robustecer esta detección, se pueden implementar reglas adicionales, como la correlación entre cuentas de origen y destino, o el análisis de segmentos por tipo de comercio (`merchant_id`) y geolocalización. Estas reglas permiten detectar patrones más complejos que podrían estar asociados a prácticas malintencionadas.

El flujo culmina con el almacenamiento y reporte de los resultados. Los datos procesados y los casos sospechosos son almacenados en bases de datos distribuidas, como AWS DynamoDB, lo que asegura su disponibilidad para auditorías o análisis futuros. Los resultados de este análisis se presentan en reportes diarios o en tiempo real, dependiendo de las necesidades del negocio. Estos reportes pueden ser visualizados mediante herramientas como Power BI, Tableau o Kibana, proporcionando a los equipos operativos una forma clara y accesible de interpretar los hallazgos y priorizar la acción.

La selección del modelo final fue fundamentada en la simplicidad, escalabilidad, interpretabilidad y posibilidad de expansión. Se optó por un enfoque inicial basado en reglas estadísticas y heurísticas, priorizando la rapidez en la implementación y la comprensión de los resultados por parte de los equipos de negocio. Este enfoque incluyó métricas claras como `rolling_count` y `rolling_sum`, que son fáciles de interpretar y utilizar en la toma de decisiones. Además, el diseño del sistema permite escalar horizontalmente, manejando eficientemente grandes volúmenes de datos mediante arquitecturas distribuidas. La solución también está diseñada para evolucionar hacia modelos avanzados de machine learning, como Random Forests o redes neuronales, utilizando los resultados validados para mejorar la precisión y reducir falsos positivos.

2. Frecuencia de Actualización de los Datos

La frecuencia de actualización de los datos depende de las necesidades del negocio, el volumen de transacciones y la criticidad del monitoreo de fraccionamientos transaccionales. En este caso, proponemos dos modalidades complementarias:

1. **Procesamiento en tiempo real:** Este enfoque es ideal para detectar y responder a posibles fraccionamientos tan pronto como ocurran. Utilizando herramientas como Apache Kafka o AWS Kinesis, se pueden procesar eventos en tiempo real para emitir alertas instantáneas cuando se detecten comportamientos sospechosos. Este enfoque es crucial para prevenir fraudes en curso y limitar posibles daños financieros o reputacionales.
2. **Procesamiento por lotes diario:** Para análisis más detallados y consolidados, se sugiere una actualización diaria que procese todas las transacciones realizadas en el día anterior. Este enfoque permite identificar patrones a mayor escala y realizar análisis más profundos que complementen la detección en tiempo real. Las herramientas como Apache Spark o Dask son ideales para procesar estos lotes de datos, dada su capacidad de manejar grandes volúmenes de información de manera eficiente.

La combinación de ambos enfoques asegura una detección rápida de transacciones malintencionadas mientras se proporciona un análisis más exhaustivo que permita optimizar los procesos y ajustar las reglas de monitoreo.

3. Arquitectura Recursos Necesarios

La arquitectura diseñada integra de manera eficiente procesamiento distribuido, almacenamiento escalable y visualización avanzada para la detección y análisis de comportamientos transaccionales malintencionados. Este diseño busca maximizar la escalabilidad y precisión en todas las etapas del flujo de datos.

La ingesta de datos en tiempo real es un componente clave para garantizar la actualización constante de la información. Tecnologías como Apache Kafka o AWS Kinesis permiten capturar y transportar datos de transacciones financieras directamente desde el Core Bancario y las plataformas de pago. Esta ingesta inicial incluye un proceso de validación para eliminar duplicados, verificar formatos y garantizar la calidad de los datos mediante herramientas como Apache Nifi o AWS Glue. La información se almacena en formatos optimizados como Parquet o Avro, permitiendo consultas rápidas y eficientes.

El procesamiento de los datos se realiza en plataformas distribuidas como Apache Spark o Dask, diseñadas para manejar grandes volúmenes de información. Este procesamiento calcula métricas críticas, como el número de transacciones (`rolling_count`), el monto acumulado (`rolling_sum`) y la desviación estándar (`rolling_std`), agrupadas por usuario, cuenta y ventanas temporales de 24 horas. Además, las transformaciones avanzadas permiten identificar patrones atípicos en las transacciones. Los resultados intermedios se almacenan en bases de datos NoSQL como AWS DynamoDB o Redis, facilitando la generación de alertas en tiempo real.

Para el almacenamiento persistente, la solución incluye bases de datos relacionales como PostgreSQL o Snowflake para almacenar datos procesados estructurados. Adicionalmente, se emplea un Data Lake como Amazon S3 o Azure Data Lake para almacenar datos históricos, asegurando la escalabilidad y optimización en el análisis a largo plazo.

La detección de transacciones sospechosas combina reglas configurables con análisis dinámico. Los umbrales pueden definirse en función de métricas como `rolling_sum > 1188.89` y condiciones como `rolling_count > 1`, ajustándose dinámicamente para mantener la precisión. Adicionalmente, se pueden incorporar modelos de clasificación basados en machine learning, como Random Forests, para calcular probabilidades de riesgo asociadas a cada grupo de transacciones.

La generación de reportes y alertas asegura la accesibilidad de los hallazgos. Tableros interactivos en herramientas como Power BI, Tableau o Kibana permiten visualizar patrones y tendencias en tiempo real. Al mismo tiempo, se implementan sistemas de alertas

mediante integraciones con Amazon SNS o PagerDuty para notificar inmediatamente sobre casos críticos.

La retroalimentación es fundamental para el mejoramiento continuo del sistema. Los casos sospechosos son revisados manualmente por equipos de monitoreo que etiquetan transacciones como fraude confirmado o falso positivo. Esta información se utiliza para ajustar umbrales y entrenar modelos predictivos avanzados, incorporando resultados validados para mejorar la precisión y reducir falsos positivos.

Finalmente, la solución está diseñada para ser escalable y segura. El uso de clústeres distribuidos en plataformas como AWS EMR o Google Dataproc permite manejar volúmenes crecientes de datos. Los datos están protegidos con cifrado en tránsito y en reposo utilizando TLS y AES-256. Además, la implementación de controles de acceso basados en roles (RBAC) y auditorías detalladas garantizan la conformidad normativa y la trazabilidad en todas las operaciones del sistema.